

CBOW Word2Vec: One Forward Pass and Detailed Backward Pass

Corpus: I like orange juice and eat eggs daily

Target word: **orange**. Window size = 2 on the left and 2 on the right. Context words: **i, like, juice, and**.

Vocabulary order: [i, like, orange, juice, and, eat, eggs, daily]. Embedding dimension N = 3. This PDF uses small made-up initial weights so the full arithmetic can be followed by hand.

Part A - Forward Pass

1. Context embeddings from W

word	dim1	dim2	dim3
i	0.10	0.20	0.30
like	0.00	0.10	0.40
juice	0.50	0.10	0.00
and	0.30	0.20	0.20

2. Average context embeddings

CBOW averages the context word vectors:

$$h = (v_i + v_{\text{like}} + v_{\text{juice}} + v_{\text{and}}) / 4$$

$$h_1 = (0.10 + 0.00 + 0.50 + 0.30) / 4 = 0.225$$

$$h_2 = (0.20 + 0.10 + 0.10 + 0.20) / 4 = 0.150$$

$$h_3 = (0.30 + 0.40 + 0.00 + 0.20) / 4 = 0.225$$

So $h = [0.225, 0.150, 0.225]$.

3. Output weight matrix W prime

dimension	i	like	orange	juice	and	eat	eggs	daily
dim 1	0.2	0.3	0.1	0.4	0.1	0.2	0.1	0.3
dim 2	0.1	0.2	0.4	0.2	0.3	0.1	0.2	0.1
dim 3	0.3	0.1	0.2	0.1	0.2	0.4	0.3	0.2

4. Compute scores / logits

Each score is a dot product between h and the column of W prime for that word.

$$\text{Example for orange: } \text{score_orange} = 0.225(0.1) + 0.150(0.4) + 0.225(0.2) = 0.1275$$

word	score z
i	0.1275
like	0.1200
orange	0.1275
juice	0.1425
and	0.1125

word	score z
eat	0.1500
eggs	0.1200
daily	0.1275

5. Softmax calculation

Softmax converts scores into probabilities: $P(\text{word}) = \exp(\text{score}) / \sum(\exp(\text{all scores}))$.

word	score z	exp(z)	probability
i	0.1275	1.13598	0.12487
like	0.1200	1.12750	0.12394
orange	0.1275	1.13598	0.12487
juice	0.1425	1.15315	0.12676
and	0.1125	1.11907	0.12302
eat	0.1500	1.16183	0.12772
eggs	0.1200	1.12750	0.12394
daily	0.1275	1.13598	0.12487

Sum of exponentials = 9.09701. For orange: $P(\text{orange}) = \exp(0.1275) / 9.09701 = 1.13598 / 9.09701 = 0.12487$.

6. Cross-entropy loss

Correct target word is orange, so the one-hot target vector is [0, 0, 1, 0, 0, 0, 0, 0].

$$\text{Loss} = -\log(P(\text{orange})) = -\log(0.12487) = 2.08045$$

Part B - Detailed Backward Pass

Backward pass means: calculate how much each weight contributed to the error, then update the weights in the opposite direction of the gradient.

1. Output error delta

For softmax + cross entropy, the derivative is simple:

$$\text{delta} = y_{\text{hat}} - y$$

Here y_{hat} is the predicted probability vector and y is the one-hot target vector.

word	prediction y_{hat}	target y	delta = $y_{\text{hat}} - y$
i	0.12487	0	0.12487
like	0.12394	0	0.12394
orange	0.12487	1	-0.87513
juice	0.12676	0	0.12676
and	0.12302	0	0.12302
eat	0.12772	0	0.12772
eggs	0.12394	0	0.12394
daily	0.12487	0	0.12487

Interpretation: orange has a large negative delta because the model predicted orange too low. The other words have positive deltas because the model gave them probability even though they were not the correct answer.

2. Gradient for W prime

Scores were computed by $z = h \times W$ prime. Therefore the gradient for W prime is:

$$dW_{\text{prime}} = h^T \times \text{delta}$$

This means each column gradient is: h multiplied by that word's delta.

For orange: $dW_{\text{prime_orange}} = [0.225, 0.150, 0.225] \times (-0.87513) = [-0.19690, -0.13127, -0.19690]$

dimension	i	like	orange	juice	and	eat	eggs	daily
dim 1	0.02810	0.02789	-0.19690	0.02852	0.02768	0.02874	0.02789	0.02810
dim 2	0.01873	0.01859	-0.13127	0.01901	0.01845	0.01916	0.01859	0.01873
dim 3	0.02810	0.02789	-0.19690	0.02852	0.02768	0.02874	0.02789	0.02810

3. Example W prime update

Using learning rate $\alpha = 0.1$:

$$W_{\text{prime_new}} = W_{\text{prime_old}} - \alpha \times dW_{\text{prime}}$$

$$\text{Orange column old} = [0.1000, 0.4000, 0.2000]$$

$$\text{Orange column gradient} = [-0.19690, -0.13127, -0.19690]$$

$$\text{Orange column new} = \text{old} - 0.1 \times \text{gradient} = [0.11969, 0.41313, 0.21969]$$

Because the orange gradient is negative, subtracting it increases the orange output weights. That helps increase $P(\text{orange})$ next time.

4. Hidden layer error: backpropagate through W prime

Now we calculate how much error flows back into the hidden vector h . Since $z = h \times W_{\text{prime}}$, we multiply by W_{prime} :

$$\text{hidden_error} = W_{\text{prime}} \times \text{delta}$$

This gives one error value for each embedding dimension. The result has shape 3×1 .

$$\text{dim 1: } 0.2(0.12487) + 0.3(0.12394) + 0.1(-0.87513) + 0.4(0.12676) + 0.1(0.12302) + 0.2(0.12772) + 0.1(0.12394) + 0.3(0.12487) = 0.11305$$

$$\text{dim 2: } 0.1(0.12487) + 0.2(0.12394) + 0.4(-0.87513) + 0.2(0.12676) + 0.3(0.12302) + 0.1(0.12772) + 0.2(0.12394) + 0.1(0.12487) = -0.20047$$

$$\text{dim 3: } 0.3(0.12487) + 0.1(0.12394) + 0.2(-0.87513) + 0.1(0.12676) + 0.2(0.12302) + 0.4(0.12772) + 0.3(0.12394) + 0.2(0.12487) = 0.02535$$

Therefore $\text{hidden_error} = [0.11305, -0.20047, 0.02535]$.

5. Gradient for each context embedding

The hidden vector h was an average of 4 context embeddings:

$$h = (v_i + v_{\text{like}} + v_{\text{juice}} + v_{\text{and}}) / 4$$

Because each context word only contributes $1/4$ of h , each context embedding gets $1/4$ of the hidden error:

$$\text{embedding_gradient} = \text{hidden_error} / 4$$

$$[0.11305, -0.20047, 0.02535] / 4 = [0.02826, -0.05012, 0.00634]$$

This same gradient is applied to each context word: i , like , juice , and and .

6. Example update for one context word embedding

Using learning rate $\alpha = 0.1$:

$$\text{embedding_new} = \text{embedding_old} - \alpha \times \text{embedding_gradient}$$

For word i :

$$\text{old } v_i = [0.10000, 0.20000, 0.30000]$$

$$\text{gradient} = [0.02826, -0.05012, 0.00634]$$

$$\text{new } v_i = [0.09717, 0.20501, 0.29937]$$

The same update pattern is used for like , juice , and and because all four context embeddings contributed equally to the average h .

Summary of the backward pass

1. $\text{delta} = \text{prediction} - \text{target}$
2. $dW_{\text{prime}} = h^T \times \text{delta}$
3. $\text{hidden_error} = W_{\text{prime}} \times \text{delta}$
4. $\text{embedding_gradient} = \text{hidden_error} / \text{number_of_context_words}$
5. Update W_{prime} and the context embeddings by subtracting $\text{learning_rate} \times \text{gradient}$.